Convex and Nonconvex Optimization

Qi(Sophie) Pan

1 Convex optimization

Convex optimization is the backbone of modern optimization. We learned some simple algorithmic schemes such as gradient descent and the Newton method among others. These two algorithms are especially suited to minimize convex functions when they are continuously differentiable or have second-order derivatives.

1. Use Newton's method to Minimize the convex function $f(x) = e^x - x$ on the entire real line $x \in \mathbb{R}$. You need to first write down the first-order and second-order derivatives of $f(x)$. Then, write down a step in the Newton's method, e.g. $x^{k+1} = x^k - f'(x^k)/f''(x^k)$. You need to plug in the expressions of the derivatives. Carry out the Newton's method from the initial point $x^0 = -1$. Write down $x^k, f(x^k), f'(x^k)$ in each iteration, until you reach a solution with the first-order derivative $|f'(x^k)| < 10^{-5}$. You should only need less than 10 steps.

   Answer:

   Let's first plot the objective function and take a look.

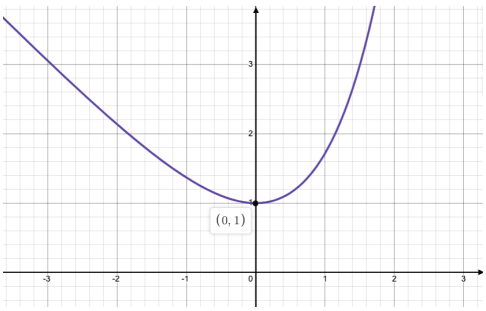

Figure 1: Plot for the objective function f(x)

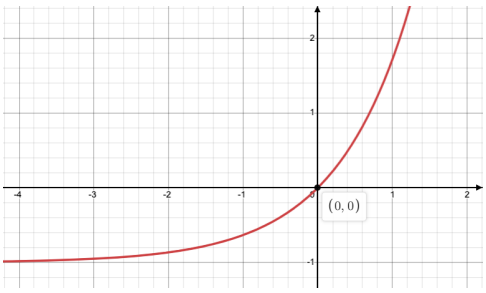

Figure 2: Plot for the 1st derivative of f(x)

We can see the min value of f(x) is at (0,1). From its 1st derivative plot, we can see when x=0, the $f'(x) = 0$. Let's use Newton's method to solve this problem.

$$f(x) = e^x - x$$
$$f'(x) = e^x - 1$$
$$f''(x) = e^x$$
$$x^{k+1} = x^k - f'(x^k)/f''(x^k) = x^k - (e^{x^k} - 1)/e^{x^k}$$

The starting point is $x^0 = -1$, we can plug -1 into all of the functions above and get the following results:

For iteration 0:

$$x^0 = -1$$
$$f(x^0) = e^{-1} + 1 = 1.3678794411714423$$
$$f'(x^0) = e^{-1} - 1 = -0.6321205588285577$$
$$f''(x^0) = e^{-1} = 0.36787944117144233$$
$$|f'(x^0)| > 10^{-5}$$

For iteration 1:

The formulas become complicated very fast, so I just write the answer here:

After hand calculating for the 1st and 2nd iterations, the process is much more tedious. Furthermore, I find we'd better not round it since deviation from rounding will accumulate and will affect the final solution value. So I change to Python and the code is shown below in figure 3.

$$x^1 = 0.7182818284590451$$
$$f(x^1) = 1.332624544233456$$
$$f'(x^1) = 1.0509063726925012$$
$$f''(x^1) = 2.050906372692501$$
$$|f'(x^1)| > 10^{-5}$$

For iteration 2:

$$x^2 = 0.20587112717830613$$
$$f(x^2) = 1.0227237341276951$$
$$f'(x^2) = 0.22859486130600137$$
$$f''(x^2) = 1.2285948613060014$$
$$|f'(x^2)| > 10^{-5}$$

For iteration 3:

$$x^3 = 0.019809091184598587$$
$$f(x^3) = 1.0001975020028975$$
$$f'(x^3) = 0.02000659318749598$$
$$f''(x^3) = 1.020006593187496$$
$$|f'(x^3)| > 10^{-5}$$

For iteration 4:

$$x^4 = 0.00019491092231630272$$
$$f(x^4) = 1.000000018996368$$
$$f'(x^4) = 0.00019492991868430565$$
$$f''(x^4) = 1.0001949299186843$$
$$|f'(x^4)| > 10^{-5}$$

For iteration 5:

$$x^5 = 1.8993899755532367e - 08$$
$$f(x^5) = 1.0000000000000002$$
$$f'(x^5) = 1.8993900008368314e - 08$$
$$f''(x^5) = 1.0000000189939$$
$$|f'(x^5)| < 10^{-5}. \text{ We stop here.}$$

```python
from math import exp
k=0 # count for the iteration
x_1=-1
control=True
while control:
    x_0=x_1
    x_1 = x_0 - (exp(x_0) - 1) / exp(x_0)  # the new point for next iteration
    f_x_0 = exp(x_0) - x_0
    f_1derivative_x_0 = exp(x_0) - 1
    f_2derivative_x_0 = exp(x_0)

    print(f'k={k}')
    print(f'x_0={x_0}')
    print(f'f_x_0 ={f_x_0}')
    print(f'f_1derivative_x_0={f_1derivative_x_0}')
    print(f'f_2derivative_x_0 ={f_2derivative_x_0}')
    print(f'x_1={x_1}')
    k += 1

    if abs(f_1derivative_x_0)>10**-5:
        control=True
    else:
        control=False
```

Figure 3: Python code for Question 1.1

2. To minimize a convex function $f(x)$ without any constraint, it is equivalent to solving the first-order optimality condition $f'(x) = 0$, i.e. find the point x^* on the curve $y = f'(x)$ that crosses the horizon line $y = 0$. Such a x^* is also called a *zero* of the equation $f'(x) = 0$. The curve $y = f'(x)$ is usually a nonlinear curve. Think of Question 1. What Newton's method actually does is to solve this nonlinear equation by solving a sequence of linear equations that approximate this nonlinear equation. How do we approximate a nonlinear curve by a linear one? Right, use the tangent line at a point of the nonlinear curve, or equivalently, use the Taylor's expansion we all know from calculus.

   So suppose we want to solve $f'(x) = 0$ and suppose the second-order derivative $f''(x)$ always exists. First, write down the tangent line of the curve $y = f'(x)$ at a point x^k. Hint: the line passing through the point $(x = a, y = b)$ with slope k has the equation $y = k(x - a) + b$.

   Suppose your equation for the tangent line is $y = k(x - a) + b$. Of course, you need to fill in k, a, b and express them in $x^k, f'(x^k), f''(x^k)$. Then, solve the equation $k(x - a) + b = 0$ and write down

the expression of the solution in k, a, b. The solution is the next iteration x^{k+1}. If you have done everything correctly, you should recover the Newton's iterate. The next step of Newton's method starts from x^{k+1}, forms the tangent line of the curve $y = f'(x)$ at x^{k+1}, and finds the zero of this linear equation, and continues.

Plot $y = f'(x)$ for $f(x) = e^x - x$. Start from $x^0 = -1$. Draw the tangent line at x^0, find the zero, and continue, until you reach x^2. You should see the same sequence as you find in the first question, and this should give you a geometric understanding of Newton's method.

Answer:

Newton's method is normally used for solving the equation which looks like $g(x) = 0$. In our optimization problem, we want to find the critical point of f(x) whose 1st derivative=0, this is such an equation that is suitable for Newton's method. The steps are listed below:

- List the function of the 1st derivative of f(x). Let's call it g(x).
- Calculate its tangent line at the start point $x^0 = -1$ which is also the 1st order Taylor approximation of g(x) at x^0. Let's denote this tangent line as $g^0(x)$.
- Solve the equation: $g^0(x) = 0$ and get the new location for x^1
- Calculate the tangent line function at x^1 and set $g^1(x) = 0$
- Solve the equation $g^1(x) = 0$ and get the new location for x^2
- Iterate for 3 times

OK, let's do it:

For iteration 0:

$f'(x) = e^x - 1$

$f''(x) = e^x$

At the point x^0 the tangent line is: $y = k(x - a) + b = f''(x^0)(x - a) + b$

when $x^0 = -1:\ a = -1,\ b = -0.6321205588285577,$

$k = f''(-1) = e^{-1} = 0.36787944117144233$

The tangent line at x^0 is:

$y = 0.36787944117144233 * x - 0.26424111765711533$

Let y=0, and solve this equation, we have:

$x^1 = 0.7182818284590451$

For iteration 1:

when $x^1 = 0.7182818284590451:\ a = 0.7182818284590451,\ b = 1.0509063726925012$

$k = f''(0.7182818284590451) = 2.050906372692501$

The tangent line at x^1 is:

$y = 2.050906372692501 * x - 0.4222224066833764$

Let y=0, and solve this equation, we have:

$x^2 = 0.2058711271783061$

For iteration 2:

when $x^2 = 0.2058711271783061:\ a = 0.2058711271783061,\ b = 0.22859486130600137$

$k = f''(0.2058711271783061) = 1.2285948613060014$

The tangent line at x^2 is:

$y = 1.2285948613060014 * x - 0.024337347636539775$

Let y=0, and solve this equation, we have:

$x^3 = 0.01980909118459854$

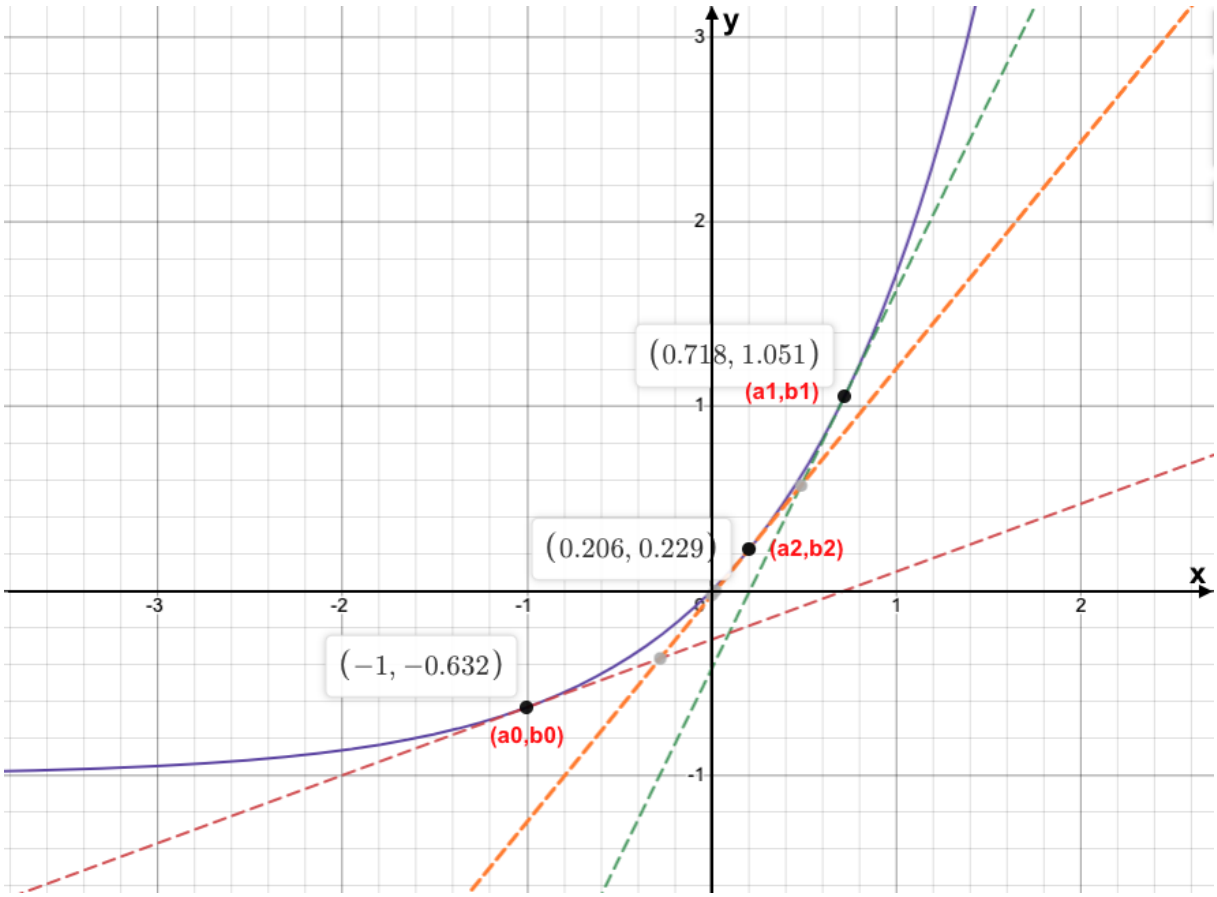

Figure 4: Plot for y=f'(x) and its tangent lines at x^0, x^1, x^2

purple line: $y = f'(x)$

red dashed line: tangent line at x^0

green dashed line: tangent line at x^1

orange dashed line: tangent line at x^2

We can see from the plot that during these 3 iterations, the new solution x for the equation is closer to 0 each time. If we keep doing it, it will converge to the origin point. This process is indeed the same as using the 2nd order Taylor approximation for question 1.1.

2 Nonconvex optimization

The following problems involve optimization of nonlinear nonconvex functions with some simple constraints, such as an interval or a box in higher dimension. To minimize each of the following functions, you can use the command `minimize` from `scipy.optimize`. Due to the nonconvex and sometimes nonsmooth (i.e. not differentiable at some points) nature of the objective function, you need to be careful about the starting point and the constraints you set. For example, you may need to set the box small enough to help the solver find a good local optimum. You need to provide a printout of your code, along with the solution.

1. $f(x_1, x_2) = (1 - x_1 + x_1x_2)^2 + (2 - x_1 + x_1^2x_2)^2 + (3 - x_1 + x_1^3x_2)^2$ over the box $-5 \leq x_1, x_2 \leq 5$. Start from $(0, 0)$. Plot the function $f(x_1, x_2)$ over the box $[-5, 5]^2$ using both a 2D contour plot and a 3D plot.

Answer:

Note: I have used Python to solve this question and all the coding and details are at the page 9.

The optimal solution is $x_1 = 1.51907761,\ x_2 = -0.27672453$.

The optimal value is 1.168412307191077.

When $-5 \leq x_1 \leq 5,\ -5 \leq x_2 \leq 5$, it forms a box shape in its 2 dimensional domain.

Note: by using different method inside *scipy.optimize.minimize()*, the optimal values are a little bit different. (difference starts from 10^{-8})

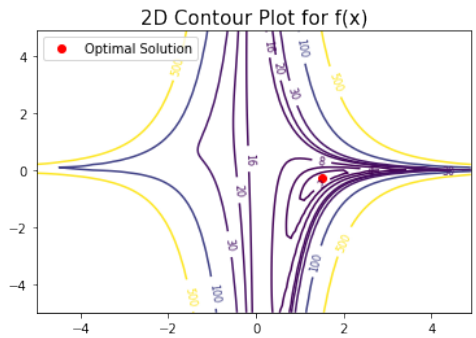

Figure 5: 2D contour plot for f(x)

From above, we can see the location of the optimal solution matches my calculation.

I have used several methods to do the plot and they look a bit different. I was shocked at the beginning as the first 3D plot looks like a bow and the 2nd one looks more organic around the optimization point.

The reason for this difference is:

Function values at the corners is much higher then everywhere else (that is what we can see in the 1st picture). That majority of blue that you can see there is not flat, but looks like that due to all of those values being significantly smaller then the corner values. In the 2nd picture, we can actually see better what is going on in that blue area.

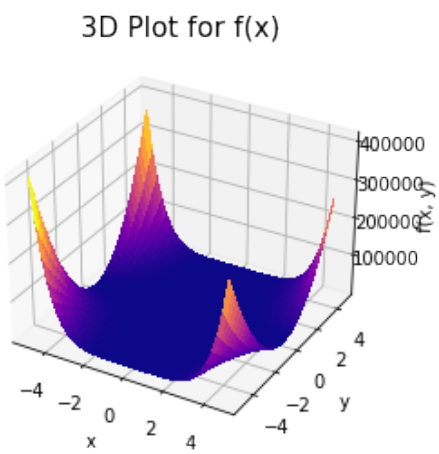

Figure 6: 3D plot 1 for f(x)

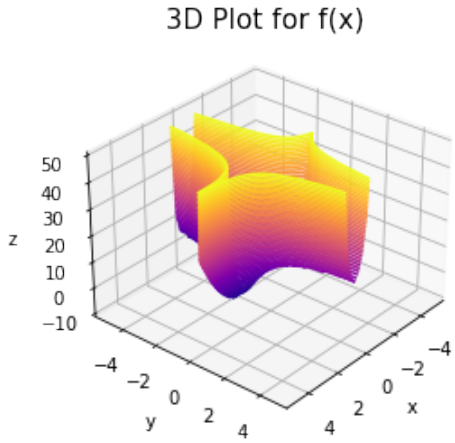

Figure 7: 3D plot 2 for f(x)

2. Consider the following function

$$f(x_1, x_2) = -0.0001 \left(\left| \sin(x_1) \sin(x_2) \exp\left(\left| 100 - \frac{\sqrt{x_1^2 + x_2^2}}{\pi} \right| \right) \right| + 1 \right)^{0.1}.$$

This function has a lot of local minima and global minima. Plot $f(x_1, x_2)$ over the box $[-10, 10]^2$ using both a 2D contour plot and a 3D plot. Try to find at least two different global minima and one local minimum that is not a global minimum. Hint: You can start the algorithm that you choose from different starting points.

Answer:

This function is very complicated and with many local and global optimal solutions. Based on my observation of the 3D and 2D plots, the global minima are near these 4 points: (1,1), (1,-1), (-1,1), (-1,-1).

Here are the plots:

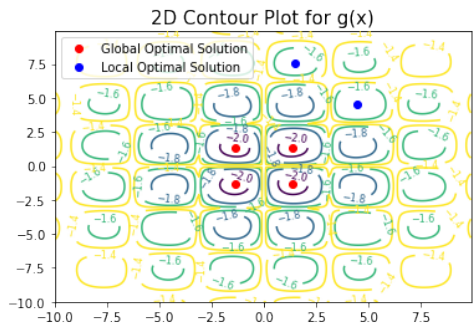

Figure 8: 2D contour plot for g(x)

Note, g(x) here is actually the f(x) in this question.

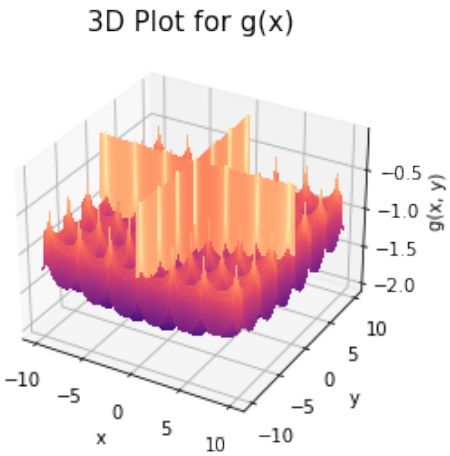

Figure 9: 3D plot for g(x)

- Start from (1,1), the optimal solution is $x_1 = 1.34938657$, $x_2 = 1.34938657$. The optimal value is -2.0626118707356023.
- Start from (1, -1), the optimal solution is $x_1 = 1.34938679$, $x_2 = -1.34938492$. The optimal value is -2.062611870729094.
- Start from (-1, -1), the optimal solution is $x_1 = -1.3493867$, $x_2 = -1.3493867$. The optimal value is -2.0626118707367493.
- Start from (-1, 1), the optimal solution is $x_1 = -1.34938492$, $x_2 = 1.34938679$. The optimal value is -2.062611870729094.

I have found 2 local minima with the starting points (5,4.8) and (1,7.5).

- Start from (5,4.8), the local optimal solution is $x_1 = 4.49100367$, $x_2 = 4.4910037$. The optimal value is -1.7906017238067369.
- Start from (1,7.5), the local optimal solution is $x_1 = 1.50849688$, $x_2 = 7.55141118$. The optimal value is -1.7154894159888365.